

Examen PF2 2024–2025 — Documents autorisés — durée 1h30

SUJET À RENDRE À LA FIN DE L'EXAMEN

1 Flux

On rappelle le type abstrait des flux ainsi que son implémentation sous forme d'une liste paresseuse :

```
module type FLUX =
sig
  type 'a t
  val vide : 'a t
  val cons : 'a -> 'a t -> 'a t
  val uncons : 'a t -> ('a * 'a t) option
  val head : 'a t -> 'a
  val tail : 'a t -> 'a t
  val unfold : ('s -> ('a * 's) option) -> 's -> 'a t
  val filter : ('a -> bool) -> 'a t -> 'a t
  val append : 'a t -> 'a t -> 'a t
  val constant : 'a -> 'a t
  val map : ('a -> 'b) -> 'a t -> 'b t
  val map2 : ('a -> 'b -> 'c) -> 'a t -> 'b t -> 'c t
  val apply : ('a -> 'b) t -> 'a t -> 'b t
  val print : int -> int t -> unit
end

type 'a flux = Tick of ('a * 'a flux) option Lazy.t

module Flux : FLUX with type 'a t = 'a flux = struct
  type 'a t = 'a flux
  ...
end
```

■ **Exercice 1:** Définir un flux des entiers naturels à partir de 2.

Pour rappel, le crible d'Ératosthène consiste à produire la liste des entiers naturels *premiers* en énumérant les entiers naturels et, pour chacun d'entre eux, en éliminant tous ses multiples du reste de la liste à venir.

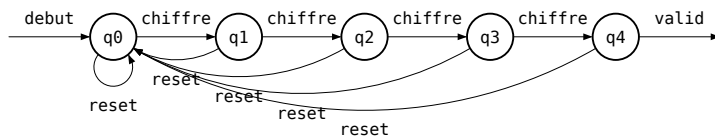
■ **Exercice 2:** Définir un flux des entiers naturels premiers en utilisant cette méthode (rappel : “n modulo h” s'écrit $n \bmod h$).

2 Typage

On cherche à décrire le comportement d'un automate bancaire lisant un code PIN à 4 chiffres. On se donne l'interface basique suivante, représentant les différentes opérations permises. Le type `t` est l'état interne de l'automate. `debut` initialise l'automate (insertion de la carte bancaire), `chiffre` ajoute un nouveau chiffre entré au clavier, `reset` réinitialise l'automate et `valid` teste si le code entré est correct, seulement si on a entré 4 chiffres à la suite.

```
module type PIN = sig
  type t
  val debut : unit -> t
  val chiffre : t -> t
  val reset : t -> t
  val valid : t -> bool
end
```

Les transitions “légales” sont données dans le schéma suivant :



On souhaite restreindre les opérations afin de ne permettre que des transitions “légales”. On ajoute au type `t` un paramètre représentant l'état interne de l'automate, soit `type _ t`.

■ **Exercice 3:** Modifier la signature des opérations de `PIN` pour n'autoriser *que les transitions légales*. (Indice : Comment représenter les états internes $q_0, \dots, q_4$ par des types?)

On souhaite maintenant mémoriser dans le type `t` les quatre derniers chiffres entrés. On se donne les nouvelles déclarations de types suivantes :

```
type ('a, 'b, 'c, 'd) t
type zero
type un
:
type neuf
type _ chfr = Zero : zero chfr | Un : un chfr | ... | Neuf : neuf chfr
```

Les paramètres `'a, ..., 'd` du type `t` sont les chiffres entrés. À l'état initial, on se comportera comme si on avait déjà entré 4 fois le chiffre '0'. On peut donc valider depuis l'état initial (et tout autre état).

■ **Exercice 4:** Modifier la signature des opérations pour modéliser ce nouveau comportement, en ajoutant à la signature de `chiffre` un argument de type `... chfr`, représentant le chiffre entré.

3 Continuations natives

En TD puis TP, vous avez étudié l'implémentation de *green threads* au moyen de la bibliothèque `Delimcc` de continuations natives, dont voici une implémentation possible :

```
module D = Delimcc

(* type d un programme pas encore lance *)
type program = unit -> unit

type res =
| Done
| Yield of continuation
| Fork of program * continuation

and continuation = unit -> res

let p : res D.prompt = D.new_prompt ()

(* fonctions presentees a l'utilisateur *)
let yield () : unit = D.shift p (fun k -> Yield k)
let fork (prog : program) : unit = D.shift p (fun k -> Fork (prog, k))
let exit () : unit = D.shift p (fun _ -> Done)

(* ordonnanceur *)
let scheduler (init_prog : program) =
  (* file des continuations (= processus en cours) *)
  let procs : continuation Queue.t = Queue.create () in
  (* démarre le programme [prog] *)
  let rec spawn (prog : program) : unit =
    loop (D.push_prompt p (fun () -> prog (); Done))
  and loop : res -> unit = function
  | Done -> if Queue.is_empty procs then () else loop (Queue.pop procs ())
  | Yield k -> Queue.push k procs; loop (Queue.pop procs ())
  | Fork (prog, k) -> Queue.push k procs; spawn prog
  in
  spawn init_prog
```

Vous allez maintenant ajouter une instruction `sleep` permettant à une tâche de se mettre en sommeil pour une durée au moins égale à un temps défini par l'utilisateur. La durée de sommeil est donnée par un flottant représentant des secondes de sommeil.

■ **Exercice 5:** Étendre le type `res` pour représenter ce nouvel effet.

■ **Exercice 6:** Définir une fonction utilisateur `sleep`: `float -> unit`.

Comment étendre l'ordonnanceur pour tenir compte de cette nouvelle fonction? Une solution naïve possible est, lorsque le *scheduler* rencontre un appel à `sleep`, de calculer la date de réveil demandée du *thread* ayant produit cet effet. Pour connaître cette dernière, vous pouvez utiliser la fonction `Sys.time` : `unit -> float` qui vous donne la “date courante” en nombre de secondes écoulées depuis le lancement du programme. Le *scheduler* peut ensuite stocker dans sa file non seulement les continuations mais aussi la date de réveil prévue pour chacune d'entre elles (0.0 pour les processus n'ayant pas produit de `sleep`).

Enfin, le *scheduler* ne réveille et relance un *thread* de sa file que si la date de réveil de ce dernier est dépassée ou nulle.

Note : cette proposition permet des attentes actives de la part du *scheduler* (si tous les *threads* dorment).

Voici un exemple de programme et d'exécution possible :

```
> let ping_pong () =
  let prog s () =
    for i = 1 to 4 do
      let sec = Random.float 0.5 in
      Format.printf "%s %d (sleep %f)@" s i sec;
      sleep sec;
      Format.printf "%s awoken@" s
    done
  in
  let ping = prog "ping" in
  let pong = prog "pong" in
  let proc () =
    fork ping;
    fork pong;
    exit ()
  in
  scheduler proc;;
> ping_pong ();;
ping 1 (sleep 0.463019)
pong 1 (sleep 0.170819)
pong awoken
ping 2 (sleep 0.298911)
ping awoken
ping 2 (sleep 0.433441)
pong awoken
pong 3 (sleep 0.393334)
pong awoken
pong 4 (sleep 0.312658)
ping awoken
ping 3 (sleep 0.045494)
ping awoken
ping 4 (sleep 0.155505)
ping awoken
pong awoken
```

■ **Exercice 7:** Modifier la fonction d'ordonnancement `scheduler` pour cette version étendue.